eff6ed43-37be-49a5-8ba3-3306df15ca

<prev_user>

<request>

You are working on a task in the 'cnfatal/rpycdec' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/rpycdec/rpycdec (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-26T02:53:53.546+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
add some comment about statement grammar to ast.py or somewahere
</problem_statement>

First, determine whether the <problem_statement> requires code changes, an explanation, a plan, or analysis. Then follow the appropriate path:

If the task requires code changes:

If the task is a question or only requires creating a plan or analysis: respond directly without making code changes.

<system_reminder>You have the capability to call multiple tools in a single response.
For maximum efficiency, whenever you need to perform multiple independent operations, ALWAYS call tools simultaneously whenever the actions can be done in parallel rather than sequentially (e.g. multiple reads/edits to different files). Especially when exploring repository, searching, reading files, viewing directories, validating changes. For example, you can read 3 different files in parallel, or edit different files in parallel. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially (e.g. reading shell output from a previous command

should be sequential as it requires the sessionID).</
system_reminder>

<prev_assistant>

Done. Added grammar docstrings to every statement AST node class in `src/renpy/ast.py` that was missing one. Each docstring follows the established style in the file and includes:

- A link to the official Ren'Py docs
- A `Grammar::` section showing the BNF/syntax pattern (using `[]` for optional parts)
- An `Examples::` section with concrete Ren'Py script snippets

Classes updated: `Say`, `Label`, `Python`, `EarlyPython`, `Image`, `Show`, `Hide`, `With`, `Return`, `If`, `Define`, `Default`, `Screen`, `TranslatePython`. Two free-standing comments that were placed *outside* their classes (`Define`, `Screen`) were also moved inside as proper docstrings.

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'cnfatal/rpycdec' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/rpycdec/rpycdec (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-05-26T03:01:28.038+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>

Fix AttributeError: 'EarlyPython' object has no attribute 'hide'
</pr_title>

<pr_description>
- [x] Survey existing docstrings in ast.py to understand current style
- [x] Add grammar docstrings to all nodes that were missing them:
  - `Say`: added grammar BNF and examples
  - `Label`: expanded with grammar BNF and `hide` example
  - `Python`: moved URL to class docstring; added grammar BNF and examples
  - `EarlyPython`: added class-level docstring with grammar BNF and examples
  - `Image`: moved docstring to class level; added grammar BNF
  - `Show`: added class docstring with grammar BNF and examples
  - `Hide`: added class docstring with grammar BNF and examples
  - `With`: added class docstring with grammar BNF and examples
  - `Return`: added class docstring with grammar BNF and examples
  - `If`: added class docstring with grammar BNF and examples
  - `Define`: moved free-standing comment into class docstring; added grammar BNF
  - `Default`: added class docstring with grammar BNF and examples
  - `Screen`: moved free-standing comment into class docstring; added grammar BNF
  - `TranslatePython`: added class docstring with grammar BNF and example
- [x] All 43 tests pass
</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_new>
<comment_id>4539567838</comment_id>
<author>@cnfatal</author>
@copilot squash commits

</comment_new>

</pr_comments>

</comments>

```
----
The last **4** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. Treat <comment_old> comments
as context only and do not make changes solely to satisfy them.
* Only act on <comment_new> comments that explicitly mention you
(@copilot). To understand the context of comments that explicitly
mention you, you may use other comments that do not explicitly
mention you. Do not act on comments that don't explicitly mention
you or reply to them, even if they are marked as <comment_new>.
* If the PR author has pushed back on or rejected a suggestion in a
thread, do not act on that thread.
* For each comment you decide to act on, determine:
    - If it is a request for changes, a question on the code, or a
suggestion.
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

<system_reminder>
You have the capability to call multiple tools in a single response.
For maximum efficiency, whenever you need to perform multiple
independent operations, ALWAYS call tools simultaneously whenever
the actions can be done in parallel rather than sequentially (e.g.
multiple reads/edits to different files). Especially when exploring
repository, searching, reading files, viewing directories,
validating changes. For example, you can read 3 different files in
parallel, or edit different files in parallel. However, if some tool
calls depend on previous calls to inform dependent values like the
parameters, do NOT call these tools in parallel and instead call
them sequentially (e.g. reading shell output from a previous command
should be sequential as it requires the sessionID).
</system_reminder>
```

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.